

A
Major Project Stage – II
Report On
Credit Card Fraud Detection

Submitted to
JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY

in partial fulfillment of the requirement
for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING (AI & ML)



SUBMITTED BY:

MOHAMMED ZAKIR AHMED	(22TRIA6637)
CHENCHU BHARATH	(22TR1A6606)
BOGA MOUNIKA	(22TR1A6605)
PACHIMATLA SAI HARSHITHA	(22TR1A6644)

Under the Guidance of
K. PRAVALIKA
ASSISTANT PROFESSOR

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (AI & ML)

SREE CHAITANYA INSTITUTE OF TECHNOLOGICAL SCIENCES
LMD COLONY, KARIMNAGAR-505527
(Approved by AICTE, Affiliated to JNTUH, Accredited by NAAC)
APRIL-2026

SREE CHAITANYA INSTITUTE OF TECHNOLOGICAL SCIENCES

LMD COLONY, KARIMNAGAR-505527

(Approved by AICTE, Affiliated to JNTUH, Accredited by NAAC)

Certificate



Certified that this **Major Project Stage-II** Report entitled, **“CREDIT CARD FRAUD DETECTION”** is the bonafide work of **MOHAMMED ZAKIR AHMED (22TR1A6637), CHENCHU BHARATH (22TR1A6606), BOGA MOUNIKA (22TR1A6605), and PACHIMATLA SAI HARSHITHA (22TR1A6644)** of IV Year, CSE (AI & ML) in the year 2026 in partial fulfillment of the requirements to award the Degree of Bachelor of Technology in **COMPUTER SCIENCE & ENGINEERING (AI & ML) Branch** of **Sree Chaitanya Institute of Technological Sciences**, Karimnagar.

K. PRAVALIKA
Assistant Professor
Project Guide

Dr. CH SAMPATH REDDY
Professor
Head of the Department

External Examiner

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our Project Guide, **Mrs. K PRAVALIKA, Assistant Professor** whose knowledge and guidance has motivated us to achieve goals we never thought possible. The time we have spent working under his supervision has truly been a pleasure.

We are thankful to **Mr. J. RAVI CHANDER, Assoc. Prof. & Project Coordinator** of CSE (AI & ML) Department for his effort, guidance and all faculty members of CSE (AI & ML) Department for their help during my course. Thanks to programmers and non-teaching staff of CSE (AI & ML) Department, Sree Chaitanya Institute of Technological Sciences.

We also thank **Dr.CH SAMPATH REDDY, HOD & Professor of CSE (AI & ML) Department** for providing seamless support and knowledge for the entire project work and also for providing right suggestions at every phase of the development of the project. He has consistently been a source of motivation, encouragement, and inspiration.

It is a great pleasure to convey our thanks to our principal **Dr. A. PRASAD RAJU, Principal**, Sree Chaitanya Institute of Technological Sciences and the College Management for permitting us to undertake this project and providing excellent facilities to carry out our project work.

Finally Special thanks to our parents, sisters and brothers for their support and encouragement throughout my life and this course. Thanks to all our friends and well wishers for their constant support.

DECLARATION

We hereby declare that the work which is being presented in this report entitled “**CREDIT CARD FRAUD DETECTION**”, submitted towards the partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in COMPUTER SCIENCE & ENGINEERING (AI & ML)**, Sree Chaitanya Institute of Technological Sciences, Karimnagar is an authentic record of our own work carried out under the supervision of **Mrs. K PRAVALIKA, Assistant Professor, Department of CSE (AI & ML)**, Sree Chaitanya Institute of Technological Sciences, Karimnagar.

To the best of our knowledge and belief, this project bears no resemblance with any report submitted to Sree Chaitanya Institute of Technological Sciences or any other University for the award of any degree or diploma.

Place: SCITS, Karimnagar
Date:

MOHAMMED ZAKIR AHMED

(22TR1A6637)

CHENCHU BHARATH

(22TR1A6606)

BOGA MOUNIKA

(22TR1A6605)

PACHIMATLA SAI HARSHITHA

(22TR1A6644)

ABSTRACT

Financial fraud, particularly credit card fraud, has become a major challenge for financial institutions due to the rapid growth of electronic and online payment systems. As credit card transactions continue to increase globally, the rate of fraudulent activities has also escalated. A critical challenge in credit card fraud detection is the highly imbalanced nature of transaction datasets, where fraudulent transactions typically represent a very small fraction of total transactions. This imbalance significantly affects the performance of traditional classification algorithms, leading to biased predictions toward legitimate transactions. Furthermore, fraudulent behavior patterns are dynamic and often resemble genuine transaction activities, making detection even more complex. To address these challenges, this paper proposes a hybrid sampling-based fraud detection framework designed to improve classification performance on highly skewed datasets. The dataset used in this study contains approximately 0.172% fraudulent transactions. The proposed system applies a hybrid sampling approach that combines oversampling of the minority (fraudulent) class and under sampling of the majority (legitimate) class to generate balanced class distributions of 34:66 and 10:90. Three supervised machine learning techniques—Naïve Bayes, k-Nearest Neighbor (KNN), and Logistic Regression—are implemented and evaluated within the proposed framework. Performance comparison is conducted using evaluation metrics suitable for imbalanced datasets, including Accuracy, Sensitivity (Recall), Specificity, Balanced Classification Rate, and Matthews Correlation Coefficient (MCC). Experimental results demonstrate that hybrid sampling significantly improves fraud detection performance compared to models trained on the original skewed dataset. The study provides a comparative analysis of the selected classifiers and highlights the importance of appropriate sampling strategies and evaluation metrics in handling highly imbalanced credit card fraud data. The proposed system contributes to improving the reliability and effectiveness of credit card fraud detection mechanisms in financial institutions.

LIST OF FIGURES

S.No	Figure Description	Page No
1	Fig:3.1: HIERARCHY IN PYTHON 3	16
2	Fig:3.2: DJANGO MVT ARCHITECTURE	28
3	Fig:3.3: DJANGO MVC FLOW DIAGRAM	29
4	Fig:3.4: HOME PAGE OF DJANGO FRAMEWORK	30
5	Fig:3.5: JUPYTER NOTEBOOK INTERFACE	34
6	Fig:3.11: ARCHITECTURE DIAGRAM	39
7	Fig:3.12: E-R DIAGRAM	40
8	Fig:3.13.1: USE CASE DIAGRAM	41
9	Fig:3.13.2: CLASS DIAGRAM	42
10	Fig:3.13.3: SEQUENCE DIAGRAM	43
11	Fig:3.13.4: ACTIVITY DIAGRAM	44
12	Fig:3.13.5: COLLABORATION DIAGRAM	45
13	Fig:5.1: HOME PAGE	48
14	Fig:5.2: CREDIT CARD DATASET UPLOAD PAGE	48
15	Fig:5.3: ML MODEL TRAINING PAGE	49
16	Fig:5.4: RANDOM FOREST RESULT PAGE	49
17	Fig:5.5: ML MODEL	50
18	Fig:5.6: TEST DATASET UPLOAD PAGE	50
19	Fig:5.7: XGBOOST ALGORITHM PAGE	51
20	Fig:5.8: FRAUD DETECTION PAGE	51

TABLE OF CONTENTS

Abstract	V
List of Figures	VI

	Page Nos Start – End
Chapter 1: INTRODUCTION	1-4
Chapter 2: PROJECT ANALYSIS	5
2.1: LITERATURE SURVEY.....	5-7
2.2: SYSTEM SPECIFICATION.....	7
2.3: EXISTING SYSTEM.....	8
2.4: PROPOSED SYSTEM.....	9
2.5: SYSTEM STUDY.....	10
2.5.1: FEASIBILITY STUDY.....	10
2.5.2: ECONOMICAL FEASIBILITY.....	10
2.5.3: TECHNICAL FEASIBILITY.....	11
2.5.4: SOCIAL FEASIBILITY.....	11
Chapter 3: PROJECT DESIGN.....	12
3.1: INPUT DESIGN.....	12
3.2: OUTPUT DESIGN.....	13
3.3: SOFTWARE ENVIRONMENT.....	14
3.3.1: PYTHON.....	14
3.3.2: PYTHON SYNTAX AND SEMANTICS.....	15-18
3.4: LIBRARIES.....	19
3.5: DEVELOPMENT ENVIRONMENTS.....	20-26

3.6: DJANGO.....	27-30
3.7: ANACONDA.....	31
3.7.1: ANACONDA NAVIGATOR.....	32
3.8: PROJECT JUPYTER.....	32
3.8.1: JUPYTER NOTEBOOK.....	33-34
3.8.2: JUPYTERHUB.....	35
3.8.3: JUPYTER LAB.....	35
3.9: PYCHARM.....	35
3.9.1: PLUGINS.....	36
3.10: MYSQL.....	36
3.10.1: MYSQL FEATURES.....	37
3.10.2: DESIGN METHODS/TYPES.....	38
3.11: ARCHITECTURE DIAGRAM.....	39
3.12: E-R DIAGRAM.....	40
3.13: UML DIAGRAMS.....	41
3.13.1: USE CASE DIAGRAM.....	41
3.13.2: CLASS DIAGRAM.....	42
3.13.3: SEQUENCE DIAGRAM.....	43
3.13.4: ACTIVITY DIAGRAM.....	44
3.13.5: COLLABORATION DIAGRAM.....	45
Chapter 4: IMPLEMENTATION.....	46-47
Chapter 5: RESULTS.....	48
5.1: HOME PAGE.....	48
5.2: CREDIT CARD DATASET UPLOAD PAGE	48

5.3: ML MODEL TRAINING PAGE	49
5.4: RANDOM FOREST RESULT PAGE	49
5.5: ML MODEL.....	50
5.6: TEST DATASET UPLOAD PAGE	50
5.7: XGBOOST ALGORITHM PAGE.....	51
5.8: FRAUD DETECTION PAGE.....	51
Chapter 6: TESTING.....	52
6.1: SYSTEM TEST.....	52
6.2: TYPES OF TESTS.....	52
6.2.1: UNIT TESTING.....	52
6.2.2: INTEGRATION TESTING.....	52
6.2.3: FUNCTIONAL TEST.....	53
6.2.4: SYSTEM TEST.....	53
6.2.5: WHITE BOX TESTING.....	53
6.2.6: BLACK BOX TESTING.....	54
6.2.7: TEST OBJECTIVES.....	54
6.2.8: ACCEPTANCE TESTING.....	54
Chapter 7: CONCLUSION.....	55
REFERENCES.....	56

CHAPTER-1

INTRODUCTION

The popularity of online shopping is growing day by day. According to an ACNielsen study conducted in 2005, one-tenth of the world's population is shopping online. Germany and Great Britain have the largest number of online shoppers, and credit card is the most popular mode of payment (59 percent). About 350 million transactions per year were reportedly carried out by Barclaycard, the largest credit card company in the United Kingdom, toward the end of the last century. Retailers like Wal-Mart typically handle much larger number of credit card transactions including online and regular purchases. As the number of credit card users rises world-wide, the opportunities for attackers to steal credit card details and, subsequently, commit fraud are also increasing. The total credit card fraud in the United States itself is reported to be \$2.7 billion in 2005 and estimated to be \$3.0 billion in 2006, out of which \$1.6 billion and \$1.7 billion, respectively, are the estimates of online fraud.

Credit-card-based purchases can be categorized into two types:

- 1) Physical card and
- 2) Virtual card.

In a physical-card-based purchase, the cardholder presents his card physically to a merchant for making a payment. To carry out fraudulent transactions in this kind of purchase, an attacker has to steal the credit card. If the cardholder does not realize the loss of card, it can lead to a substantial financial loss to the credit card company. In these cond kind of purchase, only some important information about a card (card number, expiration date, secure code) is required to make the payment. Such purchases are normally done on the Internet or over the telephone. To commit fraud in these types of purchases, a fraudster simply needs to know the card details. Most of the time, the genuine cardholder is not aware that someone else has seen or stolen his card information. The only way to detect this kind of fraud is to analyze the

spending patterns on every card and to figure out any inconsistency with respect to the “usual” spending patterns. Fraud detection based on the analysis of existing purchase data of cardholder is a promising way to reduce the rate of successful credit card frauds. Since humans tend to exhibit specific behaviouristic profiles, every cardholder can be represented by a set of patterns containing information about the typical purchase category, the time since the last purchase, the amount of money spent, etc. Deviation from such patterns is a potential threat to the system. Several techniques for the detection of credit card fraud have been proposed in the last few years.

Financial fraud is an ever growing menace with far reaching consequences in the finance industry, corporate organizations, and government. Fraud can be defined as criminal deception with intent of acquiring financial gain.

High dependence on internet technology has enjoyed increased credit card transactions. As credit card transactions become the most prevailing mode of payment for both online and offline transaction, credit card fraud rate also accelerates.

Credit card fraud can come in either inner card fraud or external card fraud. Inner card fraud occurs as a result of consent between cardholders and bank by using false identity to commit fraud while the external card fraud involves the use of stolen credit card to get cash through dubious means. A lot of researches have been devoted to detection of external card fraud which accounts for majority of credit card frauds.

Detecting fraudulent transactions using traditional methods of manual detection is time consuming and inefficient, thus the advent of big data has made manual methods more impractical. However, financial institutions have focused attention to recent computational methodologies to handle credit card fraud problem.

Data mining technique is one notable methods used in solving credit fraud detection problem. Credit card fraud detection is the process of

identifying those transactions that are fraudulent into two classes of legitimate (genuine) and fraudulent transactions. Credit card fraud detection is based on analysis of a card's spending behaviour. Many techniques have been applied to credit card fraud detection, artificial neural network, genetic algorithm, support vector machine, frequent itemset mining, decision tree, migrating birds optimization algorithm, naïve bayes. A comparative analysis of logistic regression and naïve bayes is carried out in. The performance of bayesian and neural network is evaluated on credit card fraud data.

Decision tree, neural networks and logistic regression are tested for their applicability in fraud detections.. This paper evaluates two advanced data mining approaches, support vector machines and random forests, together with logistic regression, as part of an attempt to better detect credit card fraud while neural network and logistic regression is applied on credit card fraud detection problem. A number of challenges are associated with credit card detection, namely fraudulent behaviour profile are dynamic, that is fraudulent transactions tend to look like legitimate ones; credit card transaction datasets are rarely available and highly imbalanced (or skewed); optimal feature (variables) selection for the models; suitable metric to evaluate performance of techniques on skewed credit card fraud data. Credit card fraud detection performance is greatly affected by type of sampling approach used, selection of variables and detection technique(s) used. This study investigates the effect of hybrid sampling on performance of fraud detection of naïve bayes, k-nearest neighbour and logistic regression classifiers on highly skewed credit card fraud data.

This paper seeks to carry out comparative analysis of credit card fraud detection using naïve bayes, k-nearest neighbor and logistic regression techniques on highly skewed data based on accuracy, sensitivity, specificity and Matthews's correlation coefficient (MCC) metrics. This paper extends the handling of highly imbalanced credit card fraud data in.

The imbalanced dataset used in this study which contains about 0.172% of fraud transactions is sampled in a hybrid approach. The positive class (fraud) is oversampled while the negative class (legitimate) is under-sampled by the same number of times to achieve two distributions of 34:66 and 10:90. The three techniques are applied to the data. The performance comparison of the three techniques is analyzed based on accuracy, sensitivity, specificity, Matthews Correlation Coefficient (MCC) and balanced classification rate.

CHAPTER-2

PROJECT ANALYSIS

2.1 LITERATURE SURVEY:-

Credit Card Fraud Detection:-

Credit card fraud detection has drawn a lot of research interest and a number of techniques, with special emphasis on data mining and neural networks, have been suggested. Ghosh and Reilly have proposed credit card fraud detection with a neural network. They have built a detection system, which is trained on a large sample of labelled credit card account transactions. These transactions contain example fraud cases due to lost cards, stolen cards, application fraud, counterfeit fraud, mail-order fraud, and non received issue (NRI) fraud. Recently, Syeda et al. have used parallel granular neural networks (PGNNs) for improving the speed of data mining and knowledge discovery process in credit card fraud detection. A complete system has been implemented for this purpose. Stolfo et al. suggest a credit card fraud detection system (FDS) using Meta learning techniques to learn models of fraudulent credit card transactions. Meta learning is a general strategy that provides a means for combining and integrating a number of separately built classifiers or models. A metaclassifier is thus trained on the correlation of the predictions of the base classifiers. The same group has also worked on a cost-based model for fraud and intrusion detection. They use Java agents for Meta learning (JAM), which is a distributed data mining system for credit card fraud detection. A number of important performance metrics like True Positive False Positive (TP-FP) spread and accuracy have been defined by them. Aleskerov et al. present CARDWATCH, a database mining system used for credit card fraud detection. The system, based on a neural learning module, provides an interface to a variety of commercial databases. Kim and Kim have identified skewed distribution of data and mix of legitimate and fraudulent transactions as the two main reasons for the

complexity of credit card fraud detection. Based on this observation, they use fraud density of real transaction data as a confidence value and generate the weighted fraud score to reduce the number of misdetections. Fan et al. suggest the application of distributed data mining in credit card fraud detection. Brause et al. have developed an approach that involves advanced data mining techniques and neural network algorithms to obtain high fraud coverage. Chiu and Tsai have proposed Web services and data mining techniques to establish a collaborative scheme for fraud detection in the banking industry. With this scheme, participating banks share knowledge about the fraud patterns in a heterogeneous and distributed environment. To establish a smooth channel of data exchange, Web services techniques such as XML, SOAP, and WSDL are used. Phua et al. have done an extensive survey of existing data-mining-based FDSs and published a comprehensive report. Prodromidis and Stolfo use an agent-based approach with distributed learning for detecting frauds in credit card transactions. It is based on artificial intelligence and combines inductive learning algorithms and Meta learning methods for achieving higher accuracy. Phua et al. suggest the use of metaclassifier similar to in fraud detection problems. They consider naive Bayesian C4.5, and Back Propagation neural networks as the base classifiers. A metaclassifier is used to determine which classifier should be considered based on skewness of data. Although they do not directly use credit card fraud detection as the target application, their approach is quite generic. Vatsa et al. have recently proposed a game-theoretic approach to credit card fraud detection. They model the interaction between an attacker and an FDS as a multistage game between two players, each trying to maximize his payoff. The problem with most of the abovementioned approaches is that they require labelled data for both genuine, as well as fraudulent transactions, to train the classifiers. Getting real-world fraud data is one of the biggest problems associated with credit card fraud detection. Also, these approaches cannot detect new kinds of frauds for which labelled data is not available. In contrast,

we present a Hidden Markov Model (HMM)-based credit card FDS, which does not require fraud signatures and yet is able to detect frauds by considering a cardholder's spending habit. We model a credit card transaction processing sequence by the stochastic process of an HMM. The details of items purchased in individual transactions are usually not known to an FDS running at the bank that issues credit cards to the cardholders. This can be represented as the underlying finite Markov chain, which is not observable. The transactions can only be observed through the other stochastic process that produces the sequence of the amount of money spent in each transaction. Hence, we feel that HMM is an ideal choice for addressing this problem. Another important advantage of the HMM-based approach is a drastic reduction in the number of False Positives (FPs)—transactions identified as malicious by an FDS although they are actually genuine. Since the number of genuine transactions is a few orders of magnitude higher than the number of malicious transactions, an FDS should be designed in such a way that the number of FPs is as low as possible. Otherwise, due to the “base rate fallacy” effect, bank administrators may tend to ignore the alarms. To the best of our knowledge, there is no other published literature on the application of HMM for credit card fraud detection.

2.2 SYSTEM SPECIFICATION:

HARDWARE REQUIREMENTS:

- ❖ **System** : Pentium IV 2.4 GHz.
- ❖ **Hard Disk** : 40 GB.
- ❖ **Floppy Drive** : 1.44 Mb.
- ❖ **Monitor** : 14' Colour Monitor.
- ❖ **Mouse** : Optical Mouse.
- ❖ **Ram** : 512 Mb.

SOFTWARE REQUIREMENTS:

- ❖ **Operating system** : Windows 7 Ultimate.
- ❖ **Coding Language** : Python.
- ❖ **Front-End** : Python.
- ❖ **Back-End** : Django
- ❖ **Designing** : Html, CSS, Javascript.
- ❖ **Data Base** : MySQL.

2.3 EXISTING SYSTEM

A credit card fraud detection system was proposed, which consisted of a rule-based filter, Dempster-Shafer adder, transaction history database, and Bayesian learner. The Dempster-Shafer theory combined various evidential information and created an initial belief, which was used to classify a transaction as normal, suspicious, or abnormal. If a transaction was suspicious, the belief was further evaluated using transaction history from Bayesian learning. Simulation results indicated a 98% true positive rate. A modified Fisher Discriminant function was used for credit card fraud detection. The modification made the traditional functions to become more sensitive to important instances. A weighted average was utilized to calculate variances, which allowed learning of profitable transactions. The results from the modified function confirm it can eventuate more profit.

Association rules are utilized for extracting behaviour patterns for credit card fraud cases. The data set focused on retail companies in Chile. Data samples were defuzzified and processed using the Fuzzy Query 2+ data mining tool. The resulting output reduced excessive number of rules, which simplified the task of fraud analysts. To improve the detection of credit card fraud cases, a solution was proposed. A data set from a Turkish bank was used. Each transaction was rated as fraudulent or otherwise. The misclassification rates were reduced by using the Genetic

Algorithm (GA) and scatter search. The proposed method doubled the performance, as compared with previous results.

Disadvantages

- There is no Majority Voting technique for credit card fraud detection.
- There is no Machine Learning Techniques in the existing system.

2.4 PROPOSED SYSYEM

In the proposed system, This paper investigates the performance of naïve bayes, k-nearest neighbour and logistic regression on highly skewed credit card fraud data. Dataset of credit card transactions is sourced from European cardholders containing 284,807 transactions. A hybrid technique of under-sampling and oversampling is carried out on the skewed data. The three techniques are applied on the raw and pre-processed data. The work is implemented in Python. The performance of the techniques is evaluated based on accuracy, sensitivity, specificity, precision, Matthews correlation coefficient and balanced classification rate. The results shows of optimal accuracy for naïve bayes, k-nearest neighbour and logistic regression classifiers are 97.92%, 97.69% and 54.86% respectively. The comparative results show that k-nearest neighbour performs better than naïve bayes and logistic regression techniques.

- ❖ The key contribution of this paper is the evaluation of a variety of machine learning models with a real-world credit card data set for fraud detection. While other researchers have used various methods on publicly available data sets, the data set used in this paper is extracted from actual credit card transaction information over three months.

Advantages

- The system is very fast due to Machine Learning Techniques.
- Effective Majority Classification techniques.

2.5 SYSTEM STUDY

2.5.1 FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are,

- ◆ **ECONOMICAL FEASIBILITY**
- ◆ **TECHNICAL FEASIBILITY**
- ◆ **SOCIAL FEASIBILITY**

2.5.2 ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

2.5.3 TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

2.5.4 SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

CHAPTER 3

PROJECT DESIGN

INPUT AND OUTPUT DESIGN

3.1 INPUT DESIGN

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data in to a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy. Input Design considered the following things:

- What data should be given as input?
- How the data should be arranged or coded?
- The dialog to guide the operating personnel in providing input.
- Methods for preparing input validations and steps to follow when error occur.

OBJECTIVES

1. Input Design is the process of converting a user-oriented description of the input into a computer-based system. This design is important to avoid errors in the data input process and show the correct direction to the management for getting correct information from the computerized system.

2. It is achieved by creating user-friendly screens for the data entry to handle large volume of data. The goal of designing input is to make data entry easier and to be free from errors. The data entry

screen is designed in such a way that all the data manipulates can be performed. It also provides record viewing facilities.

3. When the data is entered it will check for its validity. Data can be entered with the help of screens. Appropriate messages are provided as when needed so that the user will not be in maize of instant. Thus the objective of input design is to create an input layout that is easy to follow

3.2 OUTPUT DESIGN

A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

1. Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can use easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.

2. Select methods for presenting information.

3. Create document, report, or other formats that contain information produced by the system.

The output form of an information system should accomplish one or more of the following objectives.

- Convey information about past activities, current status or projections of the
- Future.
- Signal important events, opportunities, problems, or warnings.

- Trigger an action.
- Confirm an action.

3.3 SOFTWARE ENVIRONMENT

3.3.1 PYTHON

Python is an open source programming language that was made to be easy-to-read and powerful. A Dutch programmer named Guido van Rossum made Python in 1991. He named it after the television show Monty Python's Flying Circus.

Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language. An interpreted language, Python has a design philosophy that emphasizes code readability (notably using whitespace indentation to delimit code blocks rather than curly brackets or keywords), and a syntax that allows programmers to express concepts in fewer lines of code than might be used in languages such as C++ or Java. It provides constructs that enable clear programming on both small and large scales. Python interpreters are available for many operating systems. CPython, the reference implementation of Python, is open source software and has a community-based development model, as do nearly all of its variant implementations. CPython is managed by the non-profit Python Software Foundation. Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms.

Python was conceived in the late 1980s by Guido van Rossum at Centrum Wiskunde & Informatica (CWI) in the Netherlands as a successor to the ABC language (itself inspired by SETL), capable of exception handling and interfacing with the Amoeba operating system. Its implementation began in December 1989. Van Rossum shouldered sole responsibility for the project, as the lead developer, until 12 July 2018, when he announced his "permanent vacation" from his responsibilities as Python's Benevolent Dictator For

Life, a title the Python community bestowed upon him to reflect his long-term commitment as the project's chief decision-maker. He now shares his leadership as a member of a five-person steering council. In January 2019, active Python core developers elected Brett Cannon, Nick Coghlan, Barry Warsaw, Carol Willing and Van Rossum to a five-member "Steering Council" to lead the project. Python 2.0 was released on 16 October 2000 with many major new features, including a cycle-detecting garbage collector and support for Unicode.

Python 3.0 was released on 3 December 2008. It was a major revision of the language that is not completely backward-compatible. Many of its major features were backported to Python 2.6.x and 2.7.x version series. Releases of Python 3 include the 2to3 utility, which automates (at least partially) the translation of Python 2 code to Python 3. Python 2.7's end-of-life date was initially set at 2015 then postponed to 2020 out of concern that a large body of existing code could not easily be forward-ported to Python 3.

3.3.2 PYTHON SYNTAX AND SEMANTICS

Python is meant to be an easily readable language. Its formatting is visually uncluttered, and it often uses English keywords where other languages use punctuation. Unlike many other languages, it does not use curly brackets to delimit blocks, and semicolons after statements are optional. It has fewer syntactic exceptions and special cases than C or Pascal.

Indentation:

Python uses whitespace indentation, rather than curly brackets or keywords, to delimit blocks. An increase in indentation comes after certain statements; a decrease in indentation signifies the end of the current block. Thus, the program's visual structure accurately represents the program's semantic structure. This feature is sometimes termed the off-side rule, which some other languages share, but in most languages indentation doesn't have any semantic meaning.

Typing

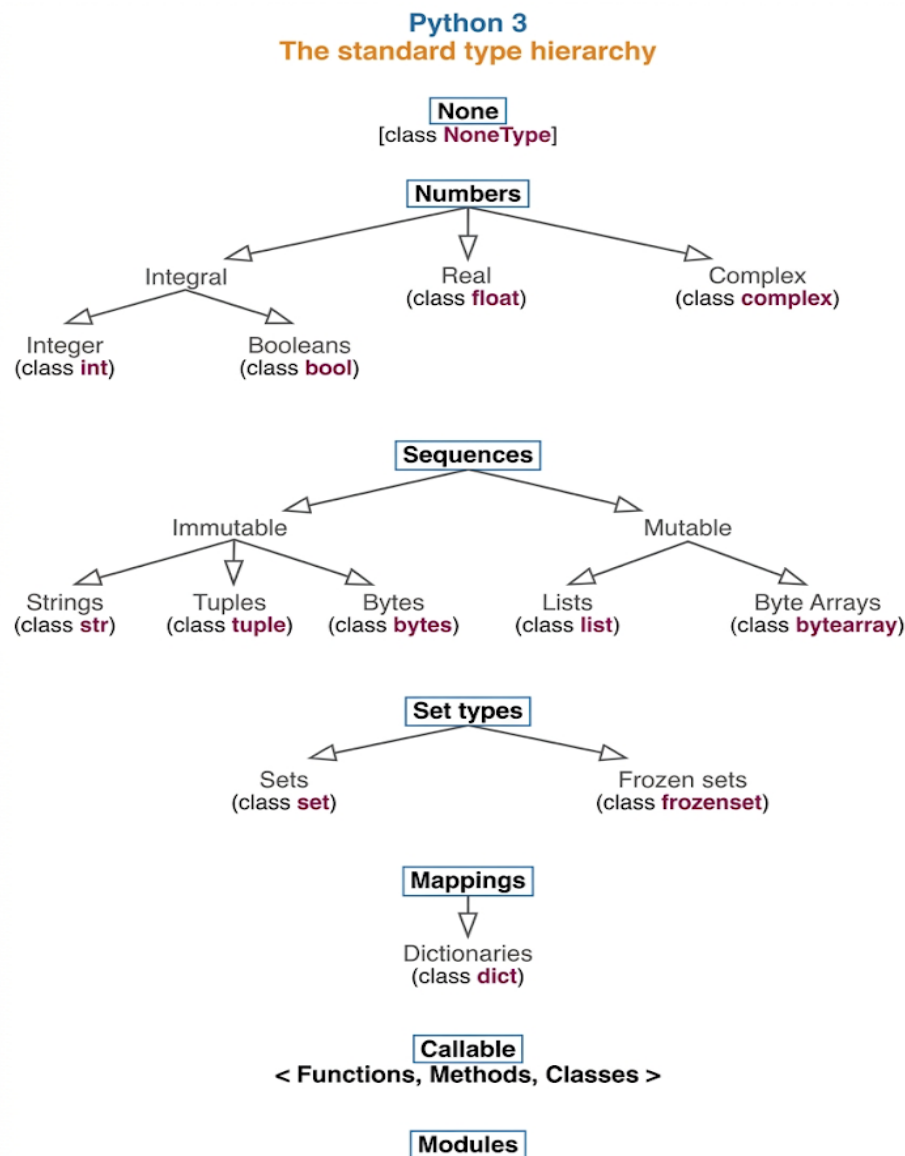


Fig 3.1 hierarchy in Python 3

Python uses duck typing and has typed objects but untyped variable names. Type constraints are not checked at compile time; rather, operations on an object may fail, signifying that the given object is not of a suitable type. Despite being dynamically typed, Python is strongly typed, forbidding operations that are not well-defined (for example, adding a number to a string) rather than silently attempting to make sense of them.

Python allows programmers to define their own types using classes, which are most often used for object-oriented programming.

New instances of classes are constructed by calling the class (for example, `SpamClass()` or `EggsClass()`), and the classes are instances of the meta class type (itself an instance of itself), allowing metaprogramming and reflection.

Before version 3.0, Python had two kinds of classes: old-style and new-style. The syntax of both styles is the same, the difference being whether the class object is inherited from, directly or indirectly (all new-style classes inherit from `object` and are instances of `type`). In versions of Python 2 from Python 2.2 onwards, both kinds of classes can be used. Old-style classes were eliminated in Python 3.0.

The long term plan is to support gradual typing and from Python 3.5, the syntax of the language allows specifying static types but they are not checked in the default implementation, C Python. An experimental optional static type checker named `mypy` supports compile-time type checking.

Mathematics

Python has the usual symbols for arithmetic operators (+, -, *, /), the floor division operator `//` and the remainder operator `%` (where the remainder can be negative, e.g. `4 % -3 == -2`). It also has `**` for exponentiation, e.g. `5**3 == 125` and `9**0.5 == 3.0`, and a matrix multiply operator `@`. [89] These operators work like in traditional math; with the same precedence rules, the operators infix (+ and - can also be unary to represent positive and negative numbers respectively). Division between integers produces floating point results. The behaviour of division has changed significantly over time:

Python 2.1 and earlier used C's division behaviour. The `/` operator is integer division if both operands are integers, and floating-point division otherwise. Integer division rounds towards 0, e.g. `7/3 == 2` and `-7/3 == -2`.

Python 2.2 changed integer division to round towards negative infinity, e.g. `7/3 == 2` and `-7/3 == -3`. The floor division `//` operator was introduced. So `7//3 == 2`, `-7//3 == -3`, `7.5//3 == 2.0` and `-7.5//3 == -`

3.0. Adding from `__future__` import `division` causes a module to use Python 3.0 rules for division (see next).

Python 3.0 changed `/` to always be floating-point division, e.g. `5/2 == 2.5`.

In Python terms, `/` is true division (or simply division), and `//` is floor division. `/` before version 3.0 is classic division.

Rounding towards negative infinity, though different from most languages, adds consistency. For instance, it means that the equation $(a + b) // b == a // b + 1$ is always true. It also means that the equation $b * (a // b) + a \% b == a$ is valid for both positive and negative values of a . However, maintaining the validity of this equation means that while the result of $a \% b$ is, as expected, in the half-open interval $[0, b)$, where b is a positive integer, it has to lie in the interval $(b, 0]$ when b is negative.

Python provides a `round` function for rounding a float to the nearest integer. For tie-breaking, Python 3 uses `round` to even: `round(1.5)` and `round(2.5)` both produce 2. Versions before 3 used round-away-from-zero: `round(0.5)` is 1.0, `round(-0.5)` is -1.0.

Python allows boolean expressions with multiple equality relations in a manner that is consistent with general use in mathematics. For example, the expression `a < b < c` tests whether a is less than b and b is less than c . C-derived languages interpret this expression differently: in C, the expression would first evaluate `a < b`, resulting in 0 or 1, and that result would then be compared with `c`.

Python uses arbitrary-precision arithmetic for all integer operations. The `Decimal` type/class in the `decimal` module provides decimal floating point numbers to a pre-defined arbitrary precision and several rounding modes. The `Fraction` class in the `fractions` module provides arbitrary precision for rational numbers.

Due to Python's extensive mathematics library, and the third-party library NumPy that further extends the native capabilities, it is

frequently used as a scientific scripting language to aid in problems such as numerical data processing and manipulation.

Python programming examples

Hello world program:

```
print('Hello, world!')
```

Program to calculate the factorial of a positive integer:

```
n = int(input('Type a number, then its factorial will be printed: '))
```

```
if n < 0:
```

```
    raise ValueError('You must enter a positive number')
```

```
fact = 1
```

```
i = 2
```

```
while i <= n:
```

```
    fact = fact * i
```

```
    i += 1
```

```
print(fact)
```

3.4 LIBRARIES

Python's large standard library, commonly cited as one of its greatest strengths, provides tools suited to many tasks. For Internet-facing applications, many standard formats and protocols such as MIME and HTTP are supported. It includes modules for creating graphical user interfaces, connecting to relational databases, generating pseudorandom numbers, arithmetic with arbitrary-precision decimals, manipulating regular expressions, and unit testing.

Some parts of the standard library are covered by specifications (for example, the Web Server Gateway Interface (WSGI) implementation [wsgiref](#) follows PEP 333), but most modules are not. They are specified by their code, internal documentation, and test suites. However, because most of the standard library is cross-platform Python code, only a few modules need altering or rewriting for variant implementations.

As of November 2019, the Python Package Index (PyPI), the official repository for third-party Python software, contains over 200,000 packages with a wide range of functionality, including:

Graphical user interfaces

Web frameworks

Multimedia

Databases

Networking

Test frameworks

Automation

Web scraping

Documentation

System administration

Scientific computing

Text processing

Image processing

Machine learning

Data analytics

3.5 DEVELOPMENT ENVIRONMENTS

Most Python implementations (including C Python) include a read-eval-print loop (REPL), permitting them to function as a command line interpreter for which the user enters statements sequentially and receives results immediately.

Other shells, including IDLE and IPython, add further abilities such as improved auto-completion, session state retention and syntax highlighting.

As well as standard desktop integrated development environments, there are Web browser-based IDEs; SageMath (intended for developing science and math-related Python programs); PythonAnywhere, a browser-based IDE and hosting environment; and Canopy IDE, a commercial Python IDE emphasizing scientific computing.

Implementations:

CPython is the reference implementation of Python. It is written in C, meeting the C89 standard with several select C99 features. It compiles Python programs into an intermediate bytecode which is then executed by its virtual machine. CPython is distributed with a large standard library written in a mixture of C and native Python. It is available for many platforms, including Windows and most modern Unix-like systems. Platform portability was one of its earliest priorities.

Other implementations

PyPy is a fast, compliant interpreter of Python 2.7 and 3.6. Its just-in-time compiler brings a significant speed improvement over CPython but several libraries written in C cannot be used with it.

Stackless Python is a significant fork of CPython that implements microthreads; it does not use the C memory stack, thus allowing massively concurrent programs. PyPy also has a stackless version.

MicroPython and CircuitPython are Python 3 variants optimized for microcontrollers. This includes Lego Mindstorms EV3.

Cross-compilers to other languages

There are several compilers to high-level object languages, with either unrestricted Python, a restricted subset of Python, or a language similar to Python as the source language:

Jython enables the use of the Java class library from a Python program.

IronPython follows a similar approach in order to run Python programs on the .NET Common Language Runtime. The RPython language can be compiled to C, and is used to build the PyPy interpreter of Python.

Pyjs compiles Python to JavaScript.

Cython compiles Python to C and C++.

Numba uses LLVM to compile Python to machine code.

Pythran compiles Python to C++.

Somewhat dated Pyrex (latest release in 2010) and Shed Skin (latest release in 2013) compile to C and C++ respectively.

Google's Grumpy compiles Python to Go.

MyHDL compiles Python to VHDL.

Nuitka compiles Python into C++.

Performance

A performance comparison of various Python implementations on a non-numerical (combinatorial) workload was presented at EuroSciPy '13.

Development

Python's development is conducted largely through the Python Enhancement Proposal (PEP) process, the primary mechanism for proposing major new features, collecting community input on issues and documenting Python design decisions. Python coding style is covered in PEP 8. Outstanding PEPs are reviewed and commented on by the Python community and the steering council.

Enhancement of the language corresponds with development of the CPython reference implementation. The mailing list python-dev is the primary forum for the language's development. Specific issues are discussed in the Roundup bug tracker hosted at bugs.python.org. Development originally took place on a self-hosted source-code repository running Mercurial, until Python moved to GitHub in January 2017. CPython's public releases come in three types, distinguished by which part of the version number is incremented:

Backward-incompatible versions, where code is expected to break and need to be manually ported. The first part of the version number is incremented. These releases happen infrequently—for example, version 3.0 was released 8 years after 2.0.

Major or "feature" releases, about every 18 months, are largely compatible but introduce new features. The second part of the version

number is incremented. Each major version is supported by bugfixes for several years after its release.

Bugfix releases, which introduce no new features, occur about every 3 months and are made when a sufficient number of bugs have been fixed upstream since the last release. Security vulnerabilities are also patched in these releases. The third and final part of the version number is incremented.

Python 3.9 alpha1 was announced in November 2019 and with the adoption of a new yearly release cadence, the first release of 3.9 is slated for November 2020.

Many alpha, beta, and release-candidates are also released as previews and for testing before final releases. Although there is a rough schedule for each release, they are often delayed if the code is not ready. Python's development team monitors the state of the code by running the large unit test suite during development, and using the BuildBot continuous integration system.

The major academic conference on Python is PyCon. There are also special Python mentoring programmes, such as Pyladies.

Naming

Python's name is derived from the British comedy group Monty Python, whom Python creator Guido van Rossum enjoyed while developing the language. Monty Python references appear frequently in Python code and culture; for example, the metasyntactic variables often used in Python literature are spam and eggs instead of the traditional foo and bar. The official Python documentation also contains various references to Monty Python routines.

The prefix Py- is used to show that something is related to Python. Examples of the use of this prefix in names of Python applications or libraries include Pygame, a binding of SDL to Python (commonly used to create games); PyQt and PyGTK, which bind Qt and GTK to Python respectively; and PyPy, a Python implementation originally written in Python.

API documentation generators

Python API documentation generators include:

Sphinx

Epydoc

HeaderDoc

pydoc

Uses

Main article: List of Python software

Since 2003, Python has consistently ranked in the top ten most popular programming languages in the TIOBE Programming Community Index where, as of February 2020, it is the third most popular language (behind Java, and C). It was selected Programming Language of the Year in 2007, 2010, and 2018.

An empirical study found that scripting languages, such as Python, are more productive than conventional languages, such as C and Java, for programming problems involving string manipulation and search in a dictionary, and determined that memory consumption was often "better than Java and not much worse than C or C++". Large organizations that use Python include Wikipedia, Google, Yahoo!, CERN, NASA, Facebook, Amazon, Instagram, Spotify and some smaller entities like ILM and ITA. The social news networking site Reddit is written entirely in Python.

Python can serve as a scripting language for web applications, e.g., via `mod_wsgi` for the Apache web server. With Web Server Gateway Interface, a standard API has evolved to facilitate these applications. Web frameworks like Django, Pylons, Pyramid, TurboGears, web2py, Tornado, Flask, Bottle and Zope support developers in the design and maintenance of complex applications. Pyjs and IronPython can be used to develop the client-side of Ajax-based applications. SQLAlchemy can be used as data mapper to a relational database. Twisted is a framework to program communications between computers, and is used (for example) by Dropbox.

Libraries such as NumPy, SciPy and Matplotlib allow the effective use of Python in scientific computing, with specialized libraries such as Biopython and Astropy providing domain-specific functionality. SageMath is a mathematical software with a notebook interface programmable in Python: its library covers many aspects of mathematics, including algebra, combinatorics, numerical mathematics, number theory, and calculus.

Python has been successfully embedded in many software products as a scripting language, including in finite element method software such as Abaqus, 3D parametric modeler like FreeCAD, 3D animation packages such as 3ds Max, Blender, Cinema 4D, Lightwave, Houdini, Maya, modo, Motion Builder, Softimage, the visual effects compositor Nuke, 2D imaging programs like GIMP, Inkscape, Scribus and Paint Shop Pro, and musical notation programs like score writer and capella. GNU Debugger uses Python as a pretty printer to show complex structures such as C++ containers. Esri promotes Python as the best choice for writing scripts in ArcGIS. It has also been used in several video games, and has been adopted as first of the three available programming languages in Google App Engine, the other two being Java and Go.

Python is commonly used in artificial intelligence projects with the help of libraries like TensorFlow, Keras, Pytorch and Scikit-learn. As a scripting language with modular architecture, simple syntax and rich text processing tools, Python is often used for natural language processing.

Many operating systems include Python as a standard component. It ships with most Linux distributions, Amiga OS 4, FreeBSD (as a package), NetBSD, OpenBSD (as a package) and macOS and can be used from the command line (terminal). Many Linux distributions use installers written in Python: Ubuntu uses the Ubiquity installer, while Red Hat Linux and Fedora use the Anaconda installer. Gentoo Linux uses Python in its package management system, Portage.

Python is used extensively in the information security industry, including in exploit development.

Most of the Sugar software for the One Laptop per Child XO, now developed at Sugar Labs, is written in Python. The Raspberry Pi single-board computer project has adopted Python as its main user-programming language.

Due to Python's user-friendly conventions and easy-to-understand language, it is commonly used as an intro language into computing sciences with students. This allows students to easily learn computing theories and concepts and then apply them to other programming languages.

LibreOffice includes Python, and intends to replace Java with Python. Its Python Scripting Provider is a core feature since Version 4.0 from 7 February 2013.

Languages influenced by Python

Python's design and philosophy have influenced many other programming languages:

Boo uses indentation, a similar syntax, and a similar object model.

Cobra uses indentation and a similar syntax, and its "Acknowledgements" document lists Python first among languages that influenced it. However, Cobra directly supports design-by-contract, unit tests, and optional static typing. Coffee Script, a programming language that cross-compiles to JavaScript, has Python-inspired syntax.

ECMAScript borrowed iterators and generators from Python. Go is designed for the "speed of working in a dynamic language like Python" and shares the same syntax for slicing arrays.

Groovy was motivated by the desire to bring the Python design philosophy to Java. Julia was designed "with true macros [.. and to be] as usable for general programming as Python [and] should be as fast as C". Calling to or from Julia is possible; to with PyCall.jl and a Python package pyjulia allows calling, in the other direction, from Python.

Kotlin is a functional programming language with an interactive shell similar to Python. However, Kotlin is statically typed with access to standard Java libraries. Nim uses indentation and a similar syntax, however it is statically typed, and offers powerful macros.[citation needed]

Ruby's creator, Yukihiro Matsumoto, has said: "I wanted a scripting language that was more powerful than Perl, and more object-oriented than Python. That's why I decided to design my own language." Swift, a programming language developed by Apple, has some Python-inspired syntax. GD Script, dynamically typed programming language used to create video-games. It is extremely similar to Python with a few minor differences.

Python's development practices have also been emulated by other languages. For example, the practice of requiring a document describing the rationale for, and issues surrounding, a change to the language (in Python, a PEP) is also used in Tcl and Erlang.

3.6 DJANGO

Django is a Python-based free and open-source web framework, which follows the model-template-view (MTV) architectural pattern. It is maintained by the Django Software Foundation (DSF), an independent organization established as a 501(c)(3) non-profit.

Django is a high-level Python Web framework that encourages rapid development and clean, pragmatic design. Built by experienced developers, it takes care of much of the hassle of Web development, so you can focus on writing your app without needing to reinvent the wheel. It's free and open source.

Django's primary goal is to ease the creation of complex, database-driven websites. Django emphasizes reusability and "pluggability" of components, rapid development, and the principle of don't repeat yourself. Python is used throughout, even for settings files and data models.

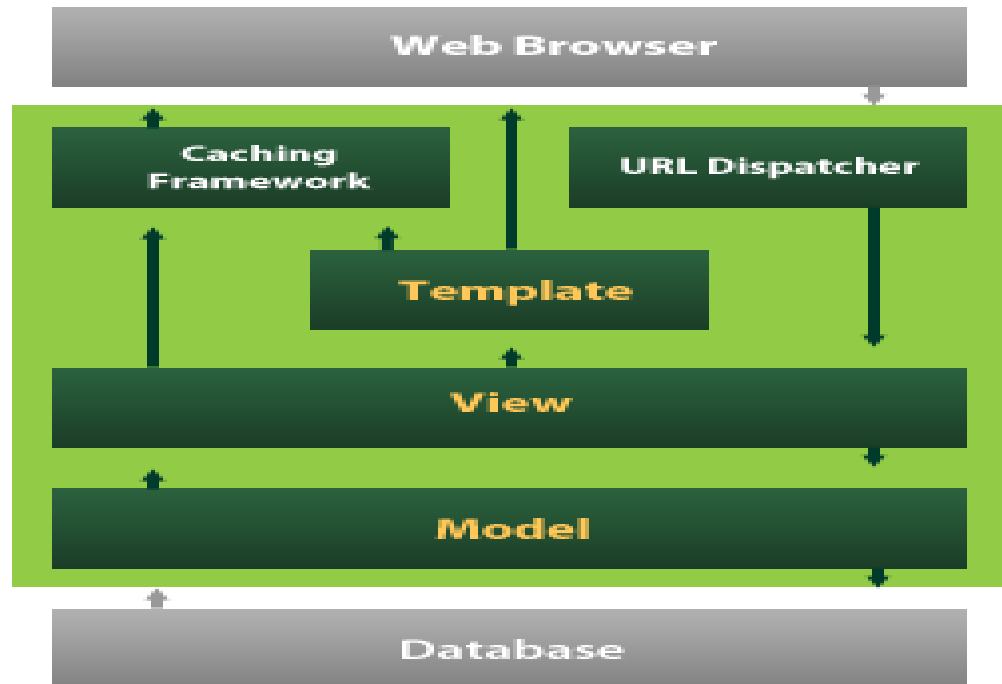


Fig 3.2 Django MVT Architecture

Django also provides an optional administrative create, read, update and delete interface that is generated dynamically through introspection and configured via admin models

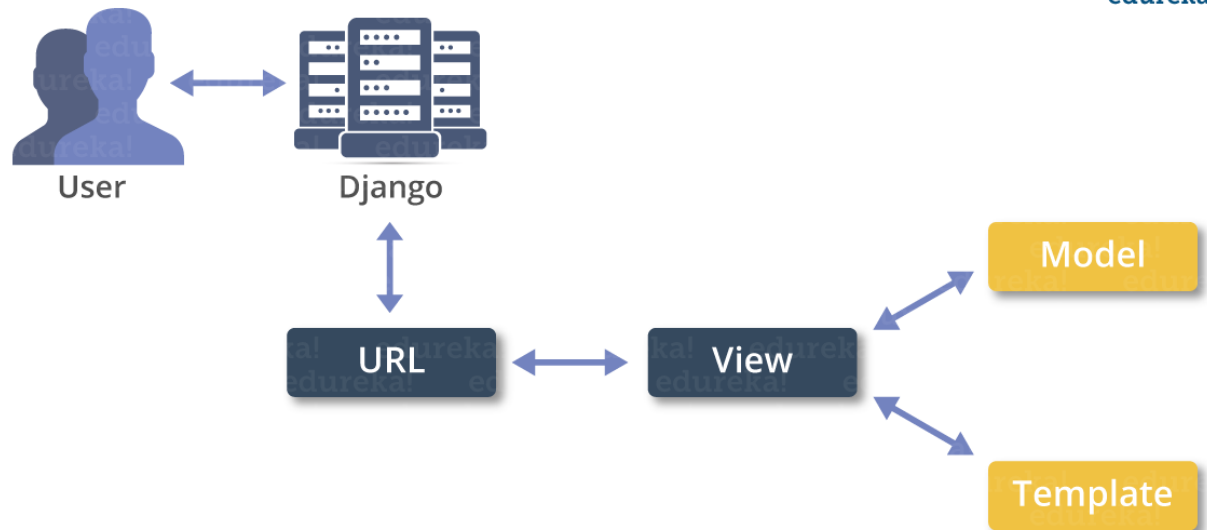


Fig3.3 Django MVC Flow Diagram

Despite having its own nomenclature, such as naming the callable objects generating the HTTP responses "views", the core Django framework can be seen as an MVC architecture. It consists of an object-relational mapper (ORM) that mediates between data models (defined as Python classes) and a relational database ("**M**odel"), a system for processing HTTP requests with a web templating system ("**V**iew"), and a regular-expression-based URL dispatcher ("**C**ontroller").

Also included in the core framework are:

- a lightweight and standalone web server for development and testing
- a form serialization and validation system that can translate between HTML forms and values suitable for storage in the database
- a template system that utilizes the concept of inheritance borrowed from object-oriented programming
- a caching framework that can use any of several cache methods
- support for middleware classes that can intervene at various stages of request processing and carry out custom functions
- an internal dispatcher system that allows components of an application to communicate events to each other via pre-defined signals
- an internationalization system, including translations of Django's own components into a variety of languages

- a serialization system that can produce and read XML and/or JSON representations of Django model instances
- a system for extending the capabilities of the template engine
- an interface to Python's built-in unit test framework
- Django REST framework is a powerful and flexible toolkit for building Web APIs.

BUNDLED APPLICATIONS

The main Django distribution also bundles a number of applications in its "contrib" package, including:

- an extensible authentication system
- the dynamic administrative interface
- tools for generating RSS and Atom syndication feeds
- a "Sites" framework that allows one Django installation to run multiple websites, each with their own content and applications
- tools for generating Google Sitemaps
- built-in mitigation for cross-site request forgery, cross-site scripting, SQL injection, password cracking and other typical web attacks, most of them turned on by default
- a framework for creating GIS applications

Home Page of Django Framework



Fig3.4 Home Page of Django Framework

3.7 ANACONDA

Anaconda is a free and open-source^[5] distribution of the Python and R programming languages for scientific computing (data science, machine learning applications, large-scale data processing, predictive analytics, etc.), that aims to simplify package management and deployment. Package versions are managed by the package management system conda.^[6] The Anaconda distribution includes data-science packages suitable for Windows, Linux, and macOS. The big difference between conda and the pip package manager is in how package dependencies are managed, which is a significant challenge for Python data science and the reason conda exists.

When pip installs a package, it automatically installs any dependent Python packages without checking if these conflict with previously installed packages^[citation needed]. It will install a package and any of its dependencies regardless of the state of the existing installation^[citation needed]. Because of this, a user with a working installation of, for example, Google Tensorflow, can find that it stops working having used pip to install a different package that requires a different version of the dependent numpy library than the one used by Tensorflow. In some cases, the package may appear to work but produce different results in detail.

In contrast, conda analyses the current environment including everything currently installed, and, together with any version limitations specified (e.g. the user may wish to have Tensorflow version 2.0 or higher), works out how to install a compatible set of dependencies, and shows a warning if this cannot be done.

Open source packages can be individually installed from the Anaconda repository^[8], Anaconda Cloud (anaconda.org), or your own private repository or mirror, using the **conda install** command. Anaconda Inc compiles and builds the packages available in the Anaconda repository itself, and provides binaries for Windows 32/64 bit, Linux 64 bit and

MacOS 64-bit. Anything available on PyPI may be installed into a conda environment using pip, and conda will keep track of what it has installed itself and what pip has installed.

Custom packages can be made using the **conda build** command, and can be shared with others by uploading them to Anaconda Cloud, PyPI or other repositories.

The default installation of Anaconda2 includes Python 2.7 and Anaconda3 includes Python 3.7. However, it is possible to create new environments that include any version of Python packaged with conda.

3.7.1 Anaconda Navigator

Anaconda Navigator is a desktop graphical user interface (GUI) included in Anaconda distribution that allows users to launch applications and manage conda packages, environments and channels without using command-line commands. Navigator can search for packages on Anaconda Cloud or in a local Anaconda Repository, install them in an environment, run the packages and update them. It is available for Windows, macOS and Linux.

The following applications are available by default in Navigator:

- JupyterLab
- Jupyter Notebook
- QtConsole
- Spyder
- Glue
- Orange
- RStudio
- Visual Studio Code

3.8 PROJECT JUPYTER

Project Jupyter is a nonprofit organization created to "develop open-source software, open-standards, and services for interactive computing across dozens of programming languages". Spun-off

from IPython in 2014 by Fernando Pérez, Project Jupyter supports execution environments in several dozen languages. Project Jupyter's name is a reference to the three core programming languages supported by Jupyter, which are Julia, Python and R, and also a homage to Galileo's notebooks recording the discovery of the moons of Jupiter. Project Jupyter has developed and supported the interactive computing products Jupyter Notebook, JupyterHub, and JupyterLab, the next-generation version of Jupyter Notebook.

3.8.1 Jupyter Notebook

Jupyter Notebook (formerly IPython Notebooks) is a web-based interactive computational environment for creating Jupyter notebook documents. The "notebook" term can colloquially make reference to many different entities, mainly the Jupyter web application, Jupyter Python web server, or Jupyter document format depending on context. A Jupyter Notebook document is a JSON document, following a versioned schema, and containing an ordered list of input/output cells which can contain code, text (using Markdown), mathematics, plots and rich media, usually ending with the ".ipynb" extension.

A Jupyter Notebook can be converted to a number of open standard output formats (HTML, presentation slides, LaTeX, PDF, ReStructuredText, Markdown, Python) through "Download As" in the web interface, via the nbconvert library or "jupyter nbconvert" command line interface in a shell. To simplify visualisation of Jupyter notebook documents on the web, the nbconvert library is provided as a service through NbViewer which can take a URL to any publicly available notebook document, convert it to HTML on the fly and display it to the user.



Fig3.5 Jupyter Notebook interface

Jupyter Notebook provides a browser-based REPL built upon a number of popular open-source libraries:

- IPython
- Tornado (web server)
- jQuery
- Bootstrap (front-end framework)
- MathJax

Jupyter Notebook can connect to many kernels to allow programming in many languages. By default Jupyter Notebook ships with the IPython kernel. As of the 2.3 release^{[8][9]} (October 2014), there are currently 49 Jupyter-compatible kernels for many programming languages, including Python, R, Julia and Haskell.

The Notebook interface was added to IPython in the 0.12 release (December 2011), renamed to Jupyter notebook in 2015 (IPython 4.0 – Jupyter 1.0). Jupyter Notebook is similar to the notebook interface of other programs such as Maple, Mathematica, and SageMath, a computational interface style that originated with Mathematica in the 1980s. According to *The Atlantic*, Jupyter interest overtook the popularity of the Mathematica notebook interface in early 2018.

3.8.2 JupyterHub

JupyterHub is a multi-user server for Jupyter Notebooks. It is designed to support many users by spawning, managing, and proxying many singular Jupyter Notebook servers. While JupyterHub requires managing servers, third-party services like JupyterLab provide an alternative to JupyterHub by hosting and managing multi-user Jupyter notebooks in the cloud.

3.8.3 Jupyter Lab

JupyterLab is the next-generation user interface for Project Jupyter. It offers all the familiar building blocks of the classic Jupyter Notebook (notebook, terminal, text editor, file browser, rich outputs, etc.) in a flexible and powerful user interface. The first stable release was announced on February 20, 2018.

3.9 PYCHARM

PyCharm is an integrated development environment (IDE) used in computer programming, specifically for the Python language. It is developed by the Czech company JetBrains. It provides code analysis, a graphical debugger, an integrated unit tester, integration with version control systems (VCSes), and supports web development with Django as well as Data Science with Anaconda.

PyCharm is cross-platform, with Windows, macOS and Linux versions. The Community Edition is released under the Apache License, and there is also Professional Edition with extra features.

Features

Coding assistance and analysis, with code completion, syntax and error highlighting, linter integration, and quick fixes

Project and code navigation: specialized project views, file structure views and quick jumping between files, classes, methods and usages

Python refactoring: includes rename, extract method, introduce variable, introduce constant, pull up, push down and others

Support for web frameworks: Django, web2py and Flask [professional edition only]

Integrated Python debugger

Integrated unit testing, with line-by-line code coverage

Google App Engine Python development [professional edition only]

Version control integration: unified user interface for Mercurial, Git, Subversion, Perforce and CVS with change lists and merge

Support for scientific tools like matplotlib, numpy and scipy [professional edition only]

It competes mainly with a number of other Python-oriented IDEs, including Eclipse's PyDev, and the more broadly focused Komodo IDE.

3.9.1 Plugins

PyCharm provides API so that developers can write their own plugins to extend PyCharm features. Several plugins from other JetBrains IDE also work with PyCharm. There are more than 1000 plugins which are compatible with PyCharm.

3.10 MySQL

A database is an other application that stores a get-together of data. MySQL is a fast easy to-use RDBMS being used for close nothing and enormous business. MySQL works using client/server design in which the server continues running on the machine containing the databases and clients interface with the server over a network. mysql is released under an open source allow. MySQL uses a standard sort of the extraordinary SQL data language. It can be handles diverse OS&with distinctive tongues including PHP, PERL, C, C++, JAVA, etc. This can be works quickly and works incredibly even with tremendous enlightening records. It sustains wide database up to 50 million parts or more in a table the default record overview keep for a table is 4GB.

3.10.1 MySQL features

1. Speed: Ofcourse, the speed at which a server side program runs depends essentially on the server equipment. Given that the server equipment is ideal, MySQL runs quick. It bolsters bunched servers for requesting applications.

2. Ease of use: MySQL is an elite, moderately basic database framework. From the earliest starting point, MySQL has commonly been arranged, observed, and overseen from the summon line. In any case, a few MySQL graphical interfaces are accessible as depicted underneath:

- **MySQL Administrator:** This apparatus makes it workable for directors to set up, assess, and tune their MySQL database server. This is planned as a trade for mysqladmin.
- **MySQL Query Browser:** Provides database designers and administrators with a graphical database task interface. It is particularly valuable for seeing various question designs and result sets in a solitary UI.
- **Configuration Wizard:** Administrators can browse a predefined rundown of ideal settings, or make their own.
- **MySQL System Tray:** Provides Windows-based chairmen a solitary perspective of their MySQL example, including the capacity to begin and stop their database servers.

3. Cost: MySQL is accessible free of cost. MySQL is an "Open Source" database. MySQL is a piece of LAMP (Linux, Apache, MySQL, PHP/Perl/Python) condition, a quickly developing open source undertaking programming stack. An ever increasing number of organizations are utilizing LAMP as an other option to costly exclusive programming stacks due to its lower cost, dependability, and documentation.

4. Query Language Support: MySQL comprehends norms based SQL (Structured Query Language).

5.Capability: Numerous customers can interface with the server in the meantime. Customers can utilize different database all the while. You

can get to MySQL utilizing a few interfaces, for example, order line customers, Web programs.

6. Connectivity and security: MySQL is completely arranged, and database can be gotten to from anyplace on the Internet, so you can impart your information to anybody, anyplace. The availability could be accomplished with Windows programs by utilizing ODBC drivers. By utilizing the ODBC connector to MySQL, any ODBC-mindful customer application (for instance, Microsoft Office, report authors, Visual Basic) can interface with MySQL.

7. Portability: MySQL keeps running on numerous assortments of UNIX, and also on other non-UNIX frameworks, for example, Windows and OS/2. MySQL keeps running on equipment from home PCs to top of the line server. MySQL can be introduced on Windows XP, Windows Server 2003, Red Hat Fedora Linux, Debian Linux, and others.

3.10.2 Design Methods / Types:-

The elements of design model are :-

1. Data Design
2. Architectural Design
3. Interface Design
4. Component-level Design

1. Data Design :-

The data design is used to transform the information domain model of analysis phase into the data structures. These data structures play an important role in software implementations. The entity relationship diagram and data dictionary are used to create the data design model. In entity relationship diagram the relationships among the data objects is defined and in data dictionary detailed data contents are given, Hence ERD and data dictionary are used to implement the data design.

2. Architectural Design :-

The architectural design is used to represent the relationship between major structural elements with the help of some “design patterns”. Hence data flow diagrams from analysis model serve as the basis for architectural design.

3. Interface Design :-

The ‘interface design’ describes how software interacts within itself. An interface means flow of information and specific type of behaviour. Hence by using the data flow and control flow diagrams the interface design can be modelled.

4. Component-level Design :-

In the ‘component-level design’ the structural elements of software architecture into procedural description of software components. Hence the component-level design can be obtained using State Transition Diagrams (STD), Control Specification (CSPEC) and Process Specification (PSPEC)

3.11 Architecture Diagram:

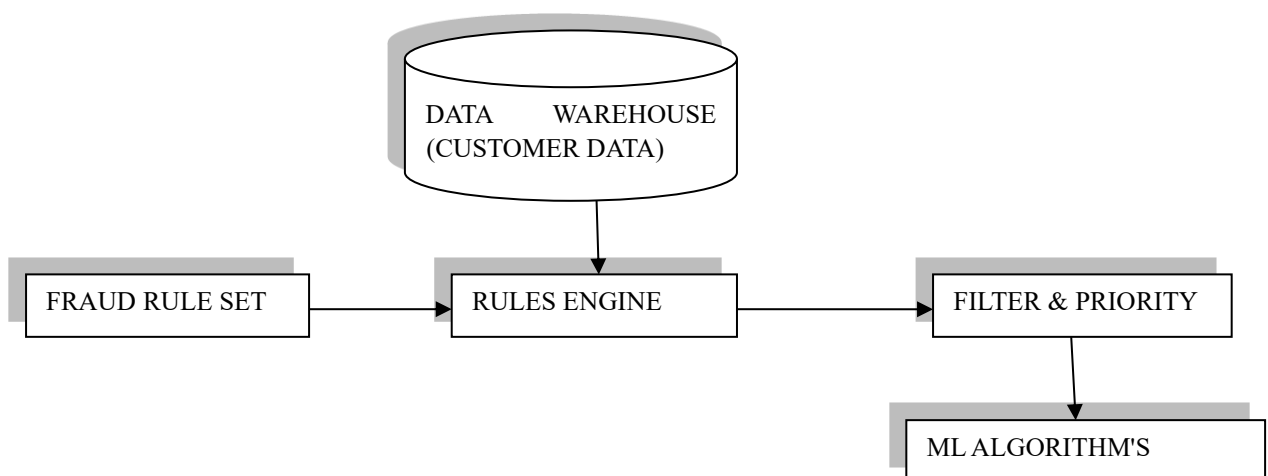


Figure 3.11 Architecture Diagram

3.12 E-R Diagram :-

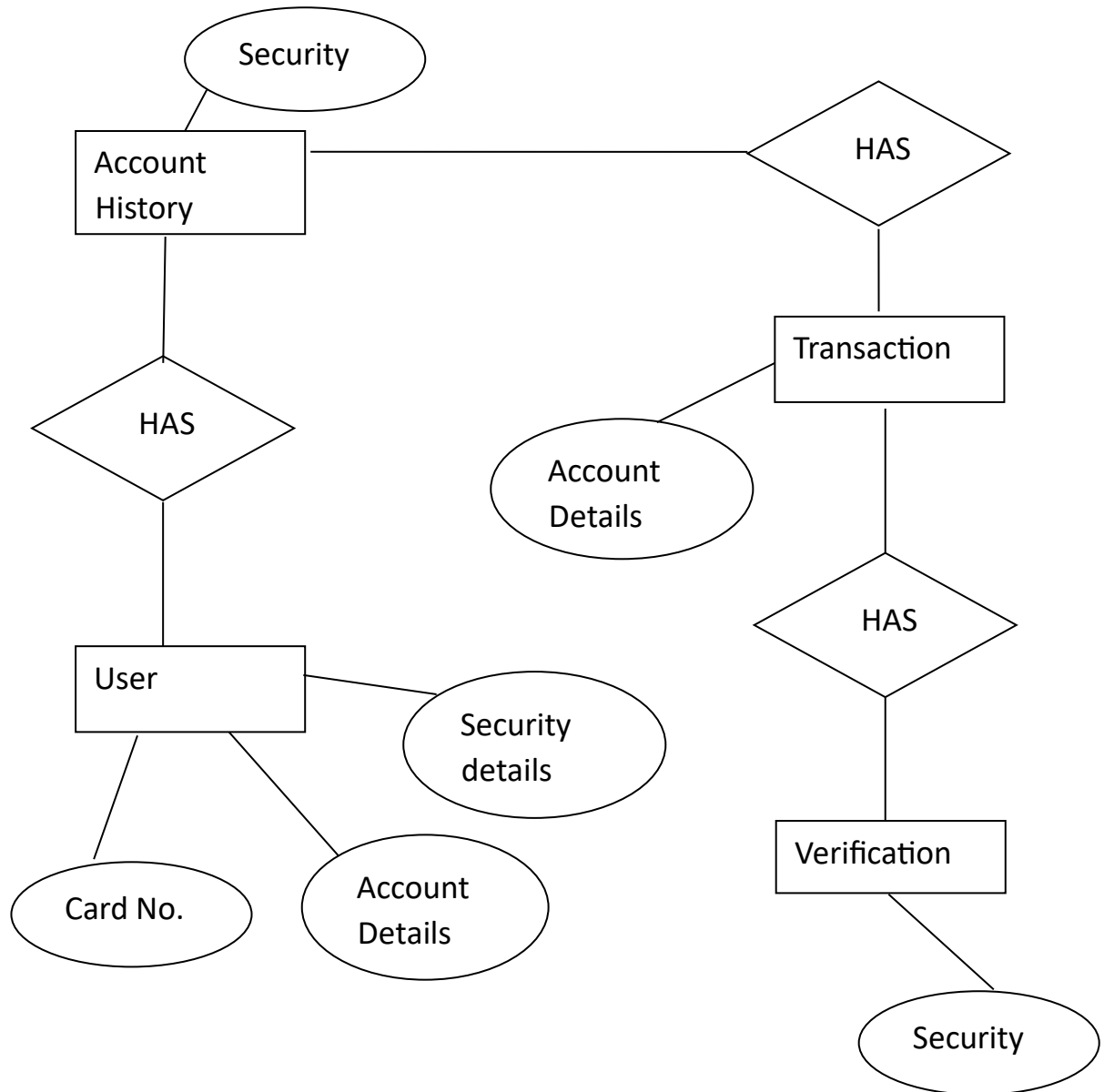


Figure 3.12 E-R Diagram

3.13 UML Diagrams :-

3.13.1 Use case diagram:

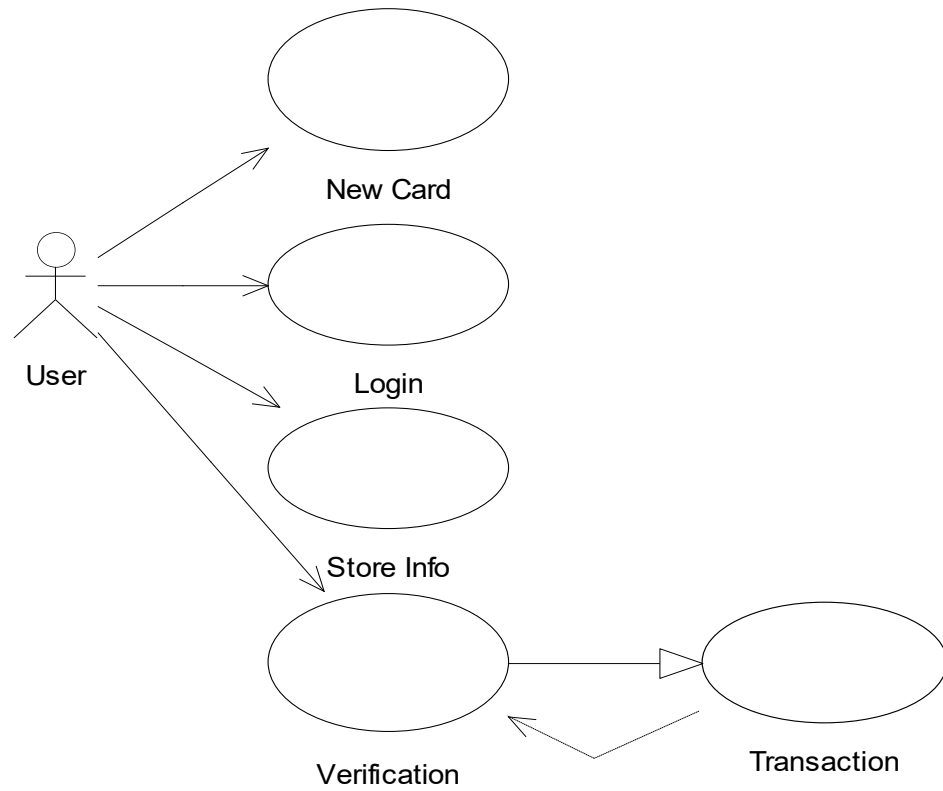


Figure 3.13.1 Use case diagram

3.13.2 Class Diagram:

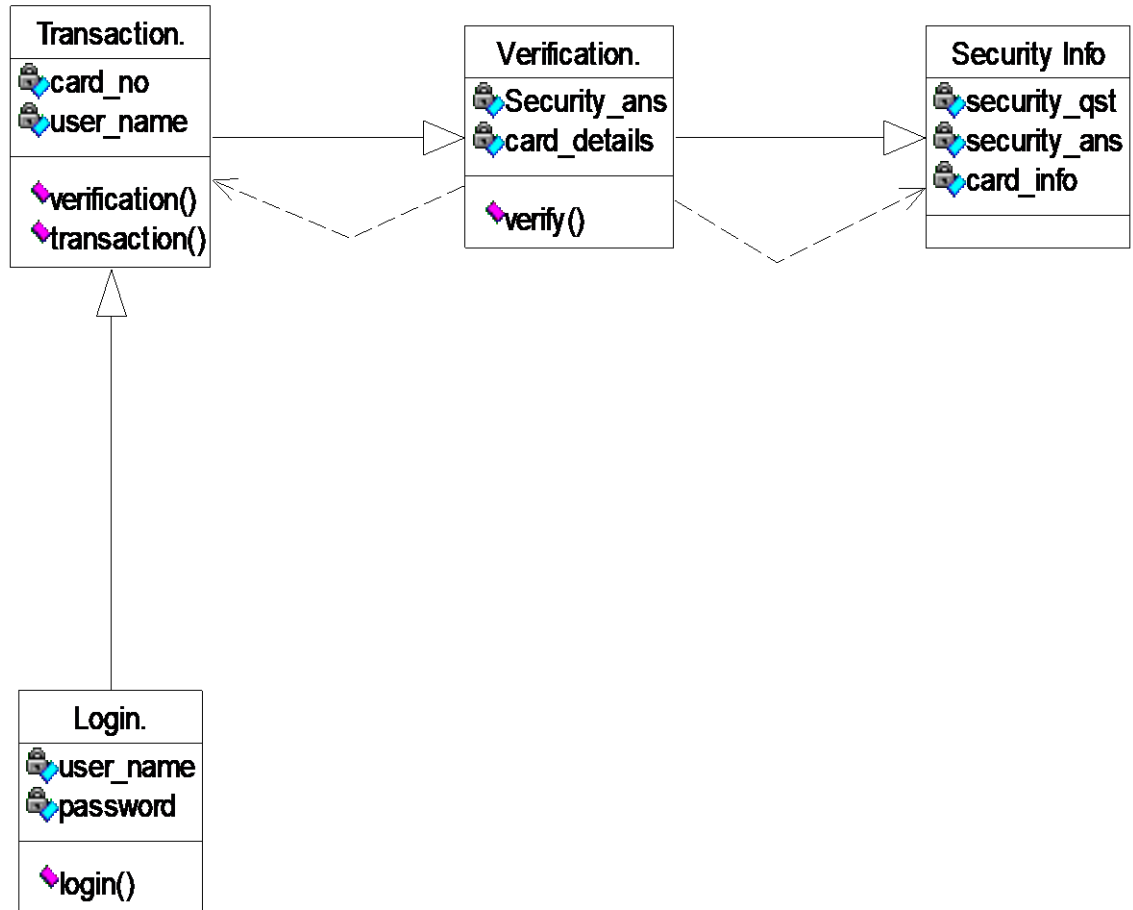


Figure 3.13.2 Class Diagram

3.13.3 Sequence Diagram:

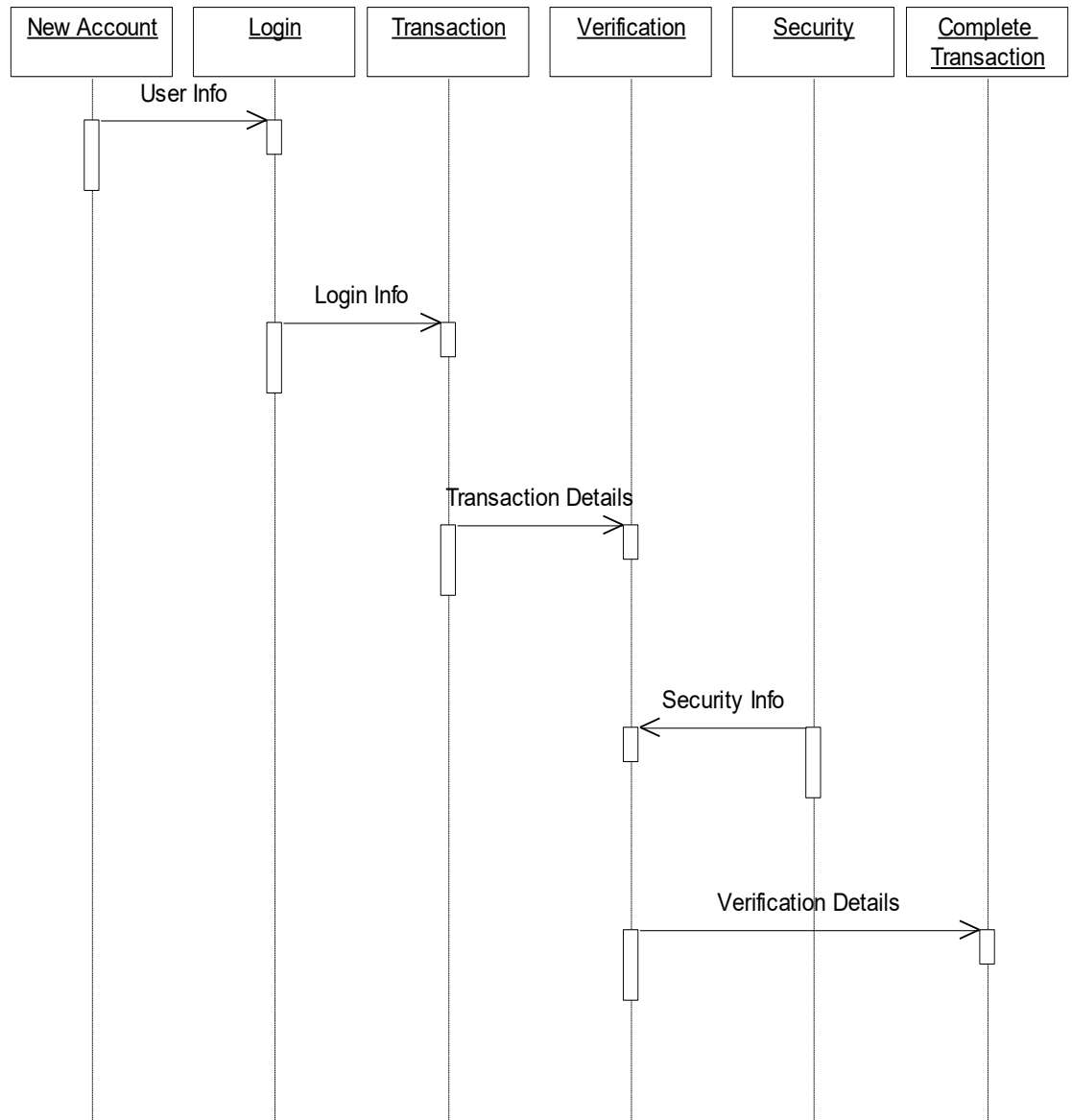


Figure 3.13.3 Sequence Diagram

3.13.4 Activity diagram:

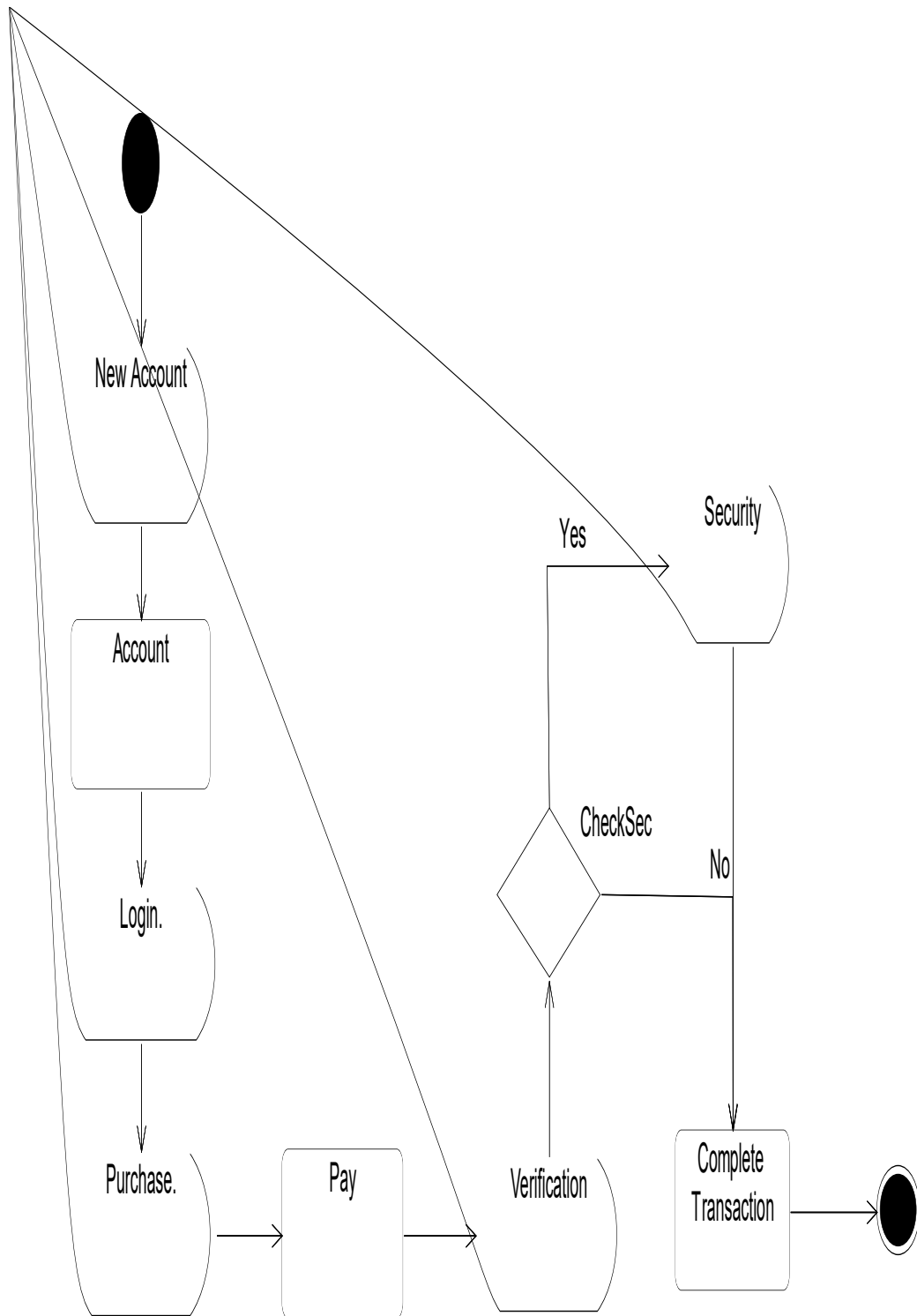


Fig 3.13.4 Activity Diagram

3.13.5 Collaboration Diagram:

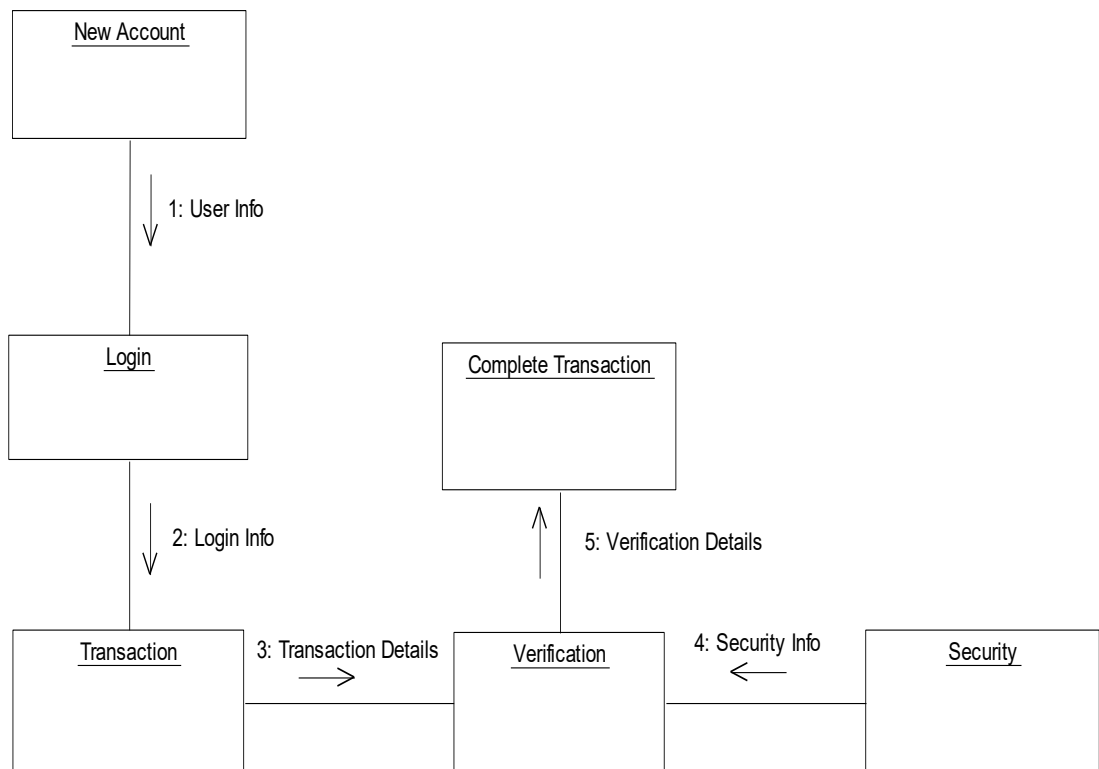


Fig:-3.13.5 Collaboration Diagram

CHAPTER 4

IMPLEMENTATION

Credit Card Fraud Detection Using Machine Learning and Deep Learning.

The implementation of the credit card fraud detection system is designed to identify fraudulent transactions by analyzing patterns in financial data using machine learning and deep learning techniques. The system processes transaction records and classifies them as either legitimate or fraudulent based on learned patterns.

Initially, the dataset is collected from financial transaction records, which typically include features such as transaction amount, time, location, merchant details, and anonymized user information. Since real-world datasets often contain imbalanced data, where fraudulent transactions are significantly fewer than legitimate ones, appropriate preprocessing techniques are applied to handle this imbalance. Missing values are addressed, duplicate entries are removed, and the data is normalized to ensure consistency.

Feature engineering is performed to extract meaningful insights from the raw data. Additional features such as transaction frequency, average spending behavior, and deviation from normal user activity are derived. These features help in identifying suspicious patterns, such as unusually high transaction amounts or transactions occurring in different geographical locations within a short time period.

For the machine learning approach, classification algorithms such as Logistic Regression, Decision Tree, and Random Forest are implemented. These models are trained using labeled transaction data to distinguish between normal and fraudulent activities. Random Forest is particularly effective due to its ability to handle complex relationships and reduce overfitting.

In addition to traditional machine learning models, a deep learning model is implemented using a feedforward neural network. The neural

network consists of multiple dense layers with activation functions such as ReLU and sigmoid, enabling it to learn complex nonlinear patterns in transaction data. Dropout layers are incorporated to improve generalization and prevent overfitting.

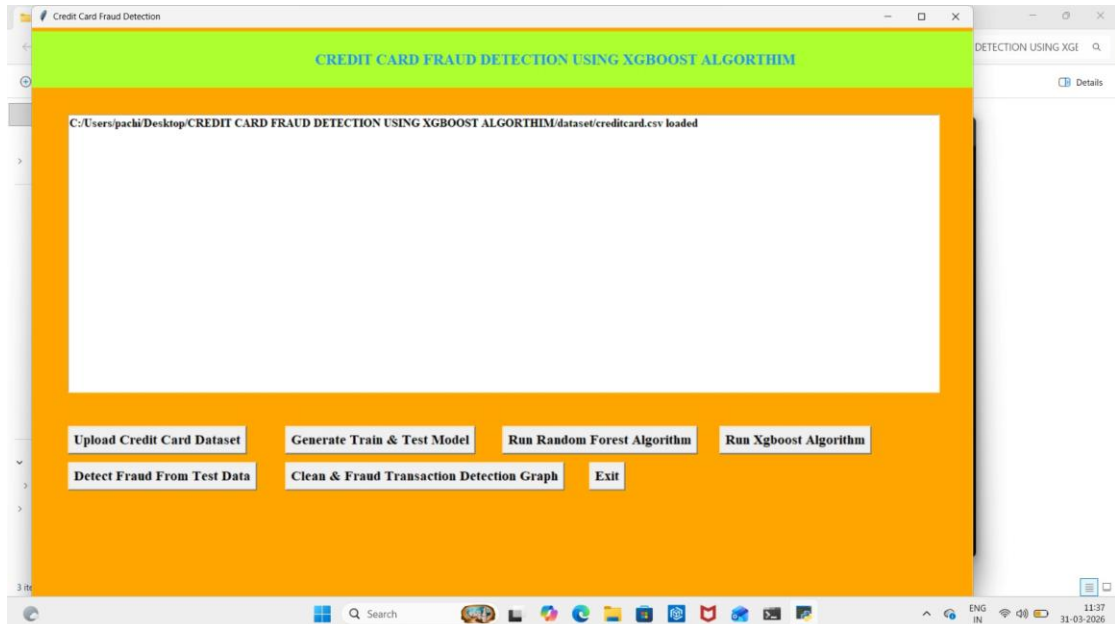
The dataset is divided into training and testing sets to evaluate the performance of the models. During training, the Binary Cross-Entropy loss function is used, and the Adam optimizer is applied to efficiently update model parameters. To further address class imbalance, techniques such as oversampling (e.g., SMOTE) or undersampling are utilized to improve the detection of minority (fraudulent) cases.

The performance of the system is evaluated using metrics such as accuracy, precision, recall, and F1-score. In fraud detection, recall is particularly important as it measures the system's ability to correctly identify fraudulent transactions. A confusion matrix is also used to analyze true positives, false positives, true negatives, and false negatives.

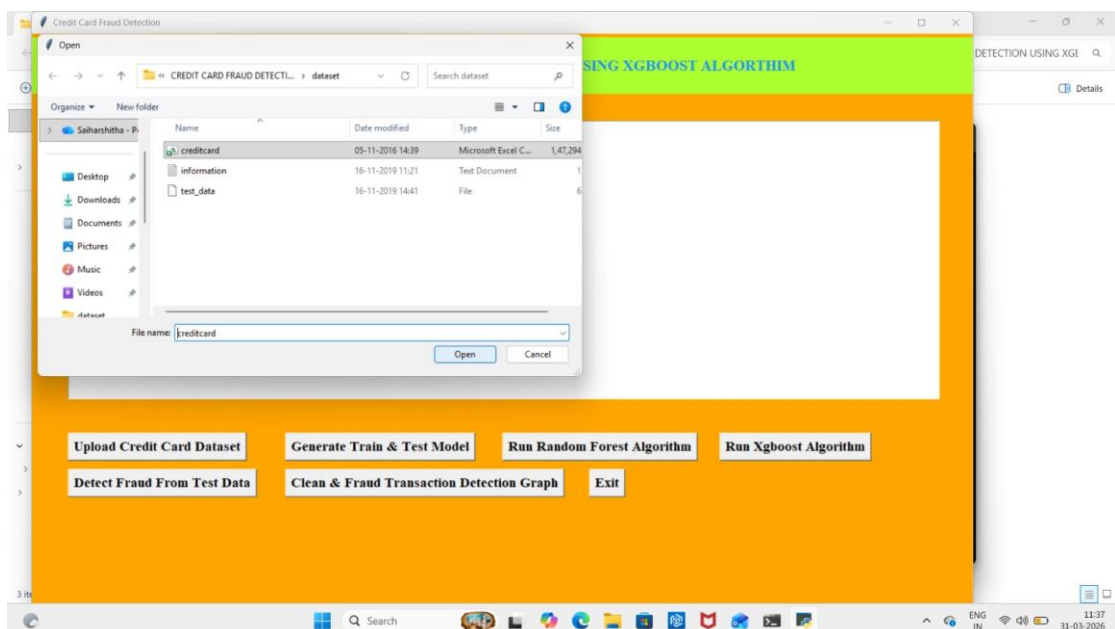
The entire system is implemented using Python programming language. Libraries such as Scikit-learn are used for machine learning models, TensorFlow or PyTorch for deep learning, and Pandas and NumPy for data preprocessing and analysis. The trained model can be deployed as a real-time monitoring system to analyze incoming transactions and flag suspicious activities.

CHAPTER-5

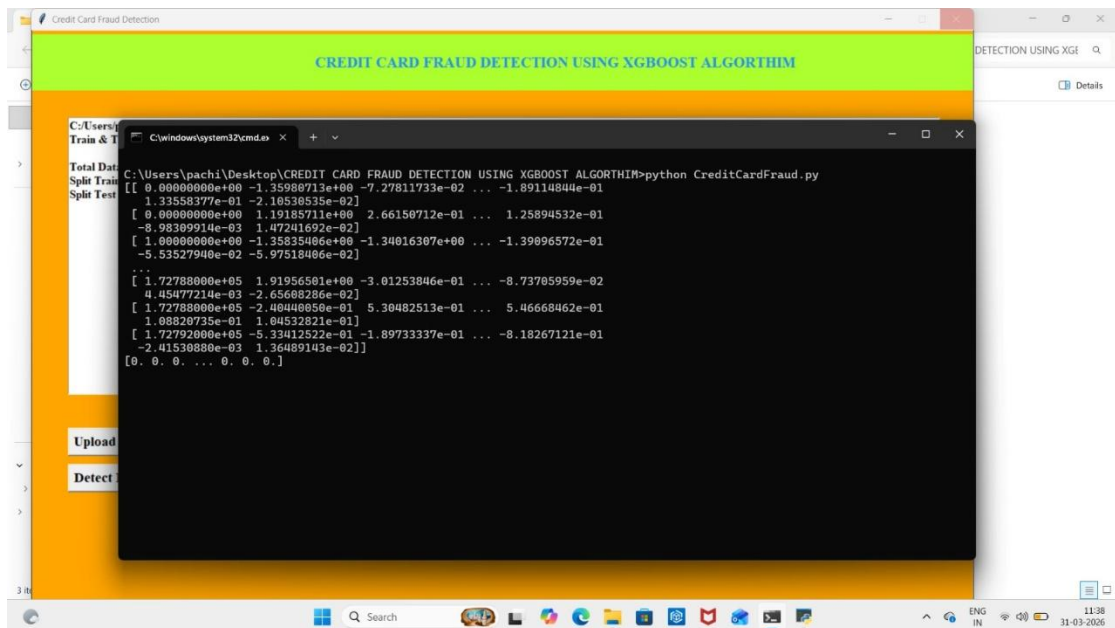
RESULTS



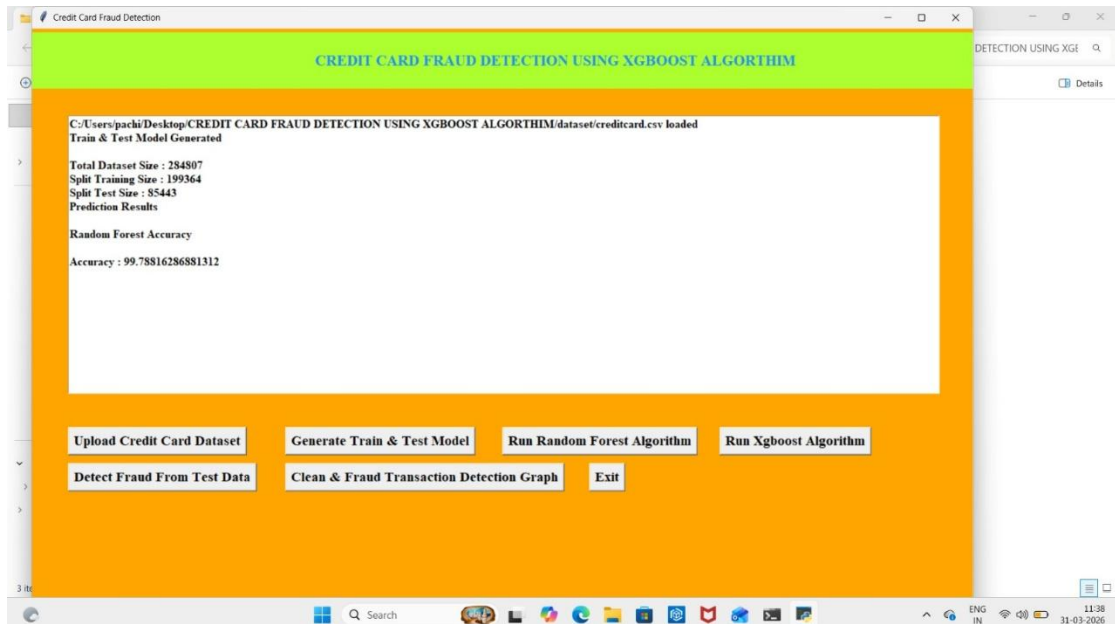
5.1 Home Page



5.2 Credit card Dataset Upload Page



5.3 ML Model Training Page



5.4 Random forest Result Page

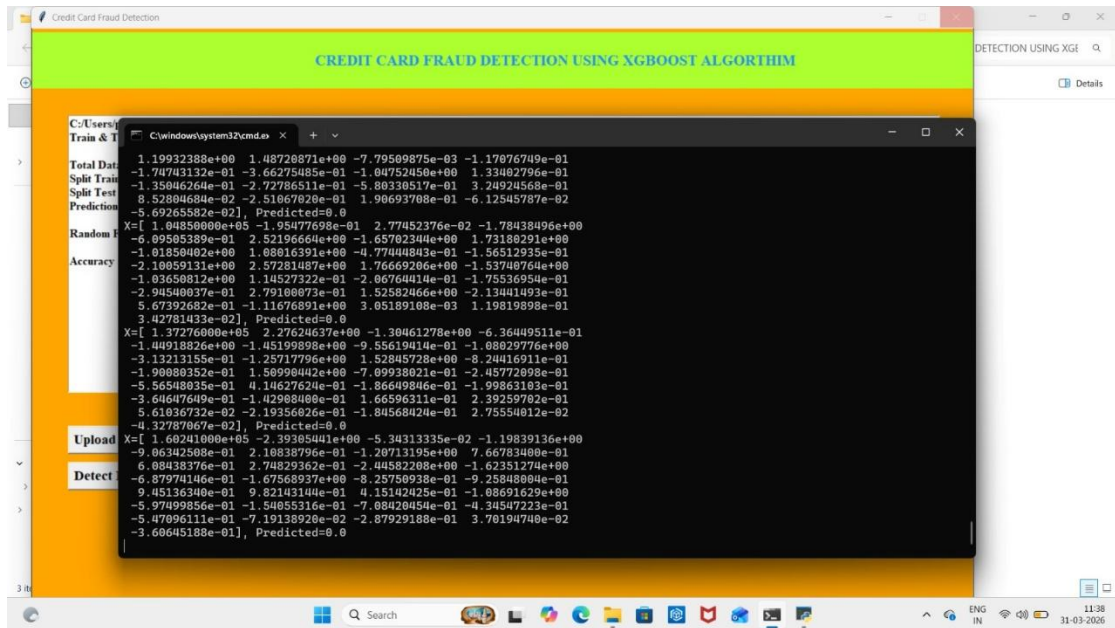


Fig 5.5 ML Model

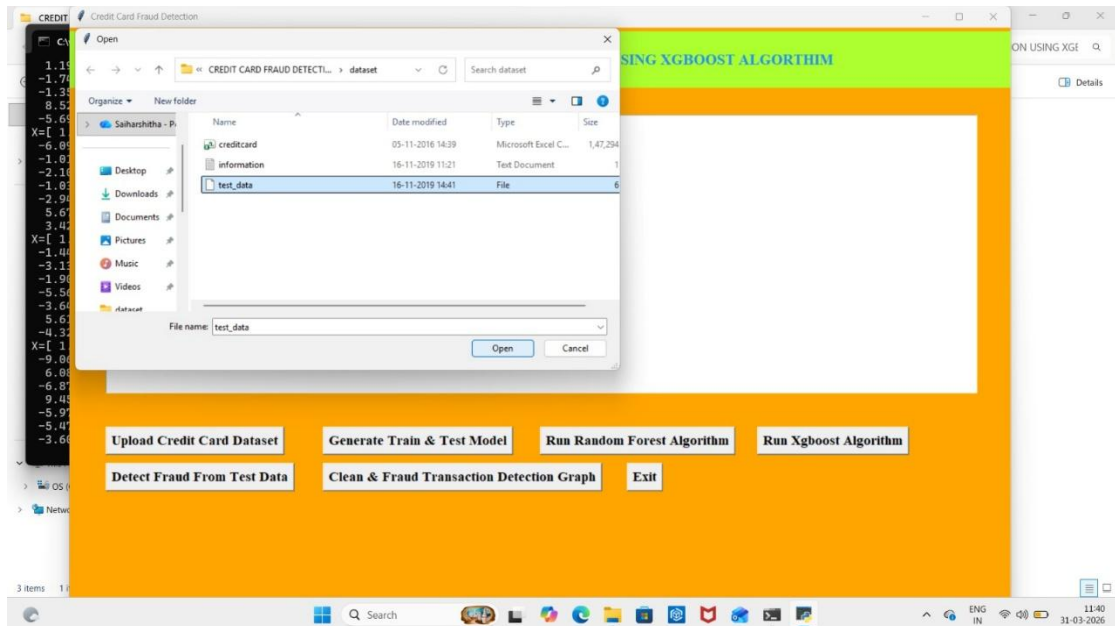


Fig 5.6 Test Dataset Upload page



Fig 5.7 Xgboost Algorithm Page

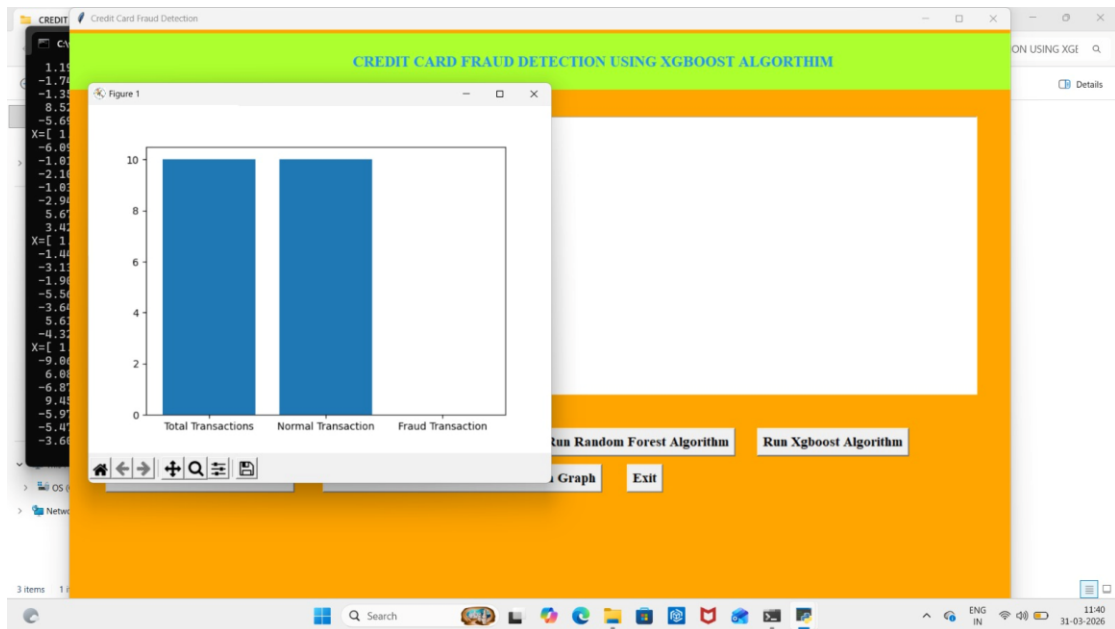


Fig 5.8 Fraud Detection Page

CHAPTER-6

TESTING

6.1 SYSTEM TEST

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

6.2 TYPES OF TESTS

6.2.1 Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

6.2.2 Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfactory, as shown by successful unit testing,

the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

6.2.3 Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output: identified classes of application outputs must be exercised.

Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

6.2.4 System Test

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

6.2.5 White Box Testing

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and

language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

6.2.6 Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

6.2.7 Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

6.2.8 Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

CHAPTER-7

CONCLUSION

In this project, a credit card fraud detection system has been successfully developed using a combination of machine learning and deep learning techniques. The system effectively analyzes transaction data to identify fraudulent activities by learning patterns and detecting anomalies in user behavior. By incorporating data preprocessing, feature engineering, and classification models, the proposed approach is able to distinguish between legitimate and fraudulent transactions with improved accuracy.

The use of machine learning algorithms such as Logistic Regression and Random Forest provides a solid baseline for classification, while the deep learning model enhances the system's ability to capture complex and non-linear relationships within the data. Additionally, techniques for handling imbalanced datasets ensure that fraudulent transactions are detected more effectively, reducing the chances of false negatives.

Performance evaluation using metrics such as accuracy, precision, recall, and F1-score demonstrates that the system is reliable and efficient in detecting fraud. The implementation also highlights the importance of recall in fraud detection scenarios, as identifying fraudulent transactions is critical for minimizing financial losses.

Overall, the developed system offers a scalable and practical solution for real-world credit card fraud detection. It can be further enhanced by integrating real-time processing, advanced deep learning architectures, and additional data sources to improve detection accuracy and adaptability. This work provides a strong foundation for building more secure and intelligent financial transaction monitoring systems.

REFERENCES

- [1] Vesta Corporation, “Credit Card Fraud Detection Using Machine Learning: A Survey,” *International Journal of Computer Applications*, vol. 182, no. 10, pp. 1–6, 2018.
- [2] Andrea Dal Pozzolo, Olivier Caelen, Yann-Aël Le Borgne, and Gianluca Bontempi, “Calibrating Probability with Undersampling for Unbalanced Classification,” in *IEEE Symposium Series on Computational Intelligence*, 2015.
- [3] Ian Goodfellow, Yoshua Bengio, and Aaron Courville, *Deep Learning*. MIT Press, 2016.
- [4] Leo Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [5] Diederik P. Kingma and Jimmy Ba, “Adam: A Method for Stochastic Optimization,” 2015.
- [6] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer, “SMOTE: Synthetic Minority Over-sampling Technique,” *Journal of Artificial Intelligence Research*, 2002.
- [7] IEEE, “IEEE Guide for Data Analytics and Machine Learning Applications,” 2020.